Microservice Architecture (/index.html)

Supported by Kong (https://konghq.com/)

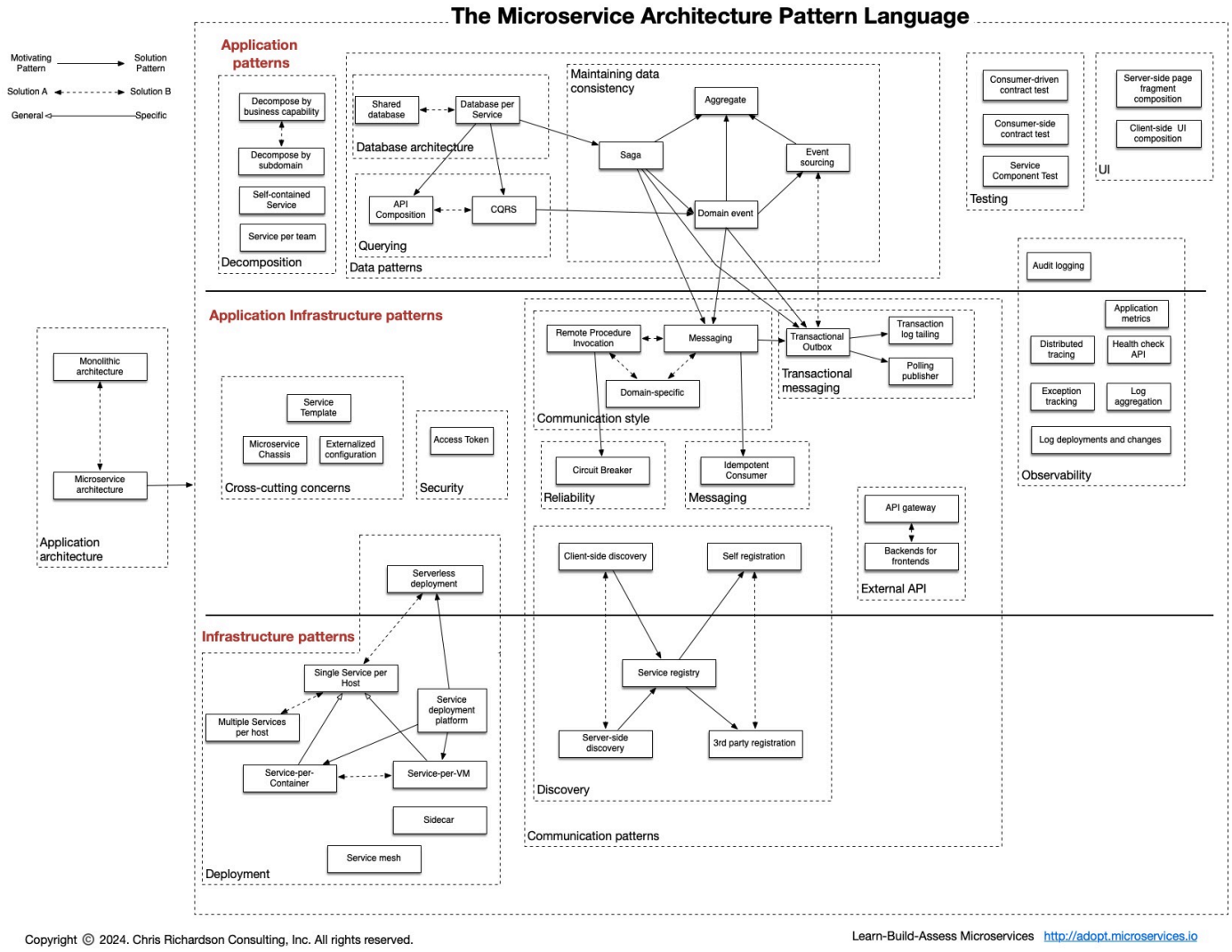
A pattern language for microservices

pattern (/tags/pattern)

The beginnings of a pattern language (http://en.wikipedia.org/wiki/Pattern_language) for microservice architectures.

点击这里，访问本系列文章的中文翻译 (cn/index.html)

Click here for Chinese translation of the patterns (cn/index.html)



Click here for PDF (../i/MicroservicePatternLanguage.pdf)

Architectural style

Which architectural style should you choose for an application?

- Monolithic architecture (monolithic.html) - architect an application as a single deployable unit
- Microservice architecture (microservices.html) - architect an application as a collection of independently deployable, loosely coupled services

Service boundaries

How to decompose an application into services?

- Decompose by business capability (decomposition/decompose-by-business-capability.html) - define services corresponding to business capabilities
- Decompose by subdomain (decomposition/decompose-by-subdomain.html) - define services corresponding to DDD subdomains
- Self-contained Service (decomposition/self-contained-service.html) - design services to handle synchronous requests without waiting for other services to respond new
- Service per team (/patterns/decomposition/service-per-team.html) new

Refactoring to services

- Strangler Application (/patterns/refactoring/strangler-application.html)
- Anti-corruption layer (/patterns/refactoring/anti-corruption-layer.html)

Service collaboration

How to implement operations that span multiple services?

- Database per Service (data/database-per-service.html) - each service has its own private database
- Shared database (data/shared-database.html) - services share a database
- Saga (data/saga.html) - use sagas, which a sequences of local transactions, to maintain data consistency across services
- Command-side replica (data/command-side-replica.html) - maintain a queryable replica of data in a service that implements a command
- API Composition (data/api-composition.html) - implement queries by invoking the services that own the data and performing an in-memory join
- CQRS (data/cqrs.html) - implement queries by maintaining one or more materialized views that can be efficiently queried
- Domain event (/patterns/data/domain-event.html) - publish an event whenever data changes
- Event sourcing (data/event-sourcing.html) - persist aggregates as a sequence of events

Transactional messaging

How to send messages as part of a database transaction?

- Transactional outbox (data/transactional-outbox.html)
- Transaction log tailing (data/transaction-log-tailing.html)
- Polling publisher (data/polling-publisher.html)

Testing

How to test services?

- Consumer-driven contract test (testing/service-integration-contract-test.html) - a test suite for a service that is written by the developers of another service that consumes it
- Consumer-side contract test (testing/service-integration-contract-test.html) - a test suite for a service client (e.g. another service) that verifies that it can communicate with the service
- Service component sest (testing/service-component-test.html) - a test suite that tests a service in isolation using test doubles for any services that it invokes

Deployment

How to deploy an application's services?

- Multiple service instances per host (deployment/multiple-services-per-host.html) - deploy multiple service instances on a single host
- Service instance per host (deployment/single-service-per-host.html) - deploy each service instance in its own host
- Service instance per VM (deployment/service-per-vm.html) - deploy each service instance in its VM
- Service instance per Container (deployment/service-per-container.html) - deploy each service instance in its container
- Serverless deployment (/patterns/deployment/serverless-deployment.html) - deploy a service using serverless deployment platform
- Service deployment platform (/patterns/deployment/service-deployment-platform.html) - deploy services using a highly automated deployment platform that provides a service abstraction

Cross-cutting concerns

How to handle cross cutting concerns?

- Microservice chassis (/patterns/microservice-chassis.html) - a framework that handles cross-cutting concerns and simplifies the development of services
- Externalized configuration (/patterns/externalized-configuration.html) - externalize all configuration such as database location and credentials
- Service Template (/patterns/service-template.html) - a template that implements standard cross cutting concerns and is intended to be copied by a developer in order to quickly start developing a new service

Communication styles

Which mechanisms do services use to communicate with each other and their external clients?

- Remote Procedure Invocation (communication-style/rpi.html) - use an RPI-based protocol for inter-service communication
- Messaging (communication-style/messaging.html) - use asynchronous messaging for inter-service communication
- Domain-specific protocol (communication-style/domain-specific.html) - use a domain-specific protocol
- Idempotent Consumer (/patterns/communication-style/idempotent-consumer.html) - ensure that message consumers can cope with being invoked multiple times with the same message

External API

How do external clients communicate with the services?

- API gateway (apigateway.html) - a service that provides each client with unified interface to services
- Backend for front-end (apigateway.html) - a separate API gateway for each kind of client

Service discovery

How does the client of an RPI-based service discover the network location of a service instance?

- Client-side discovery (client-side-discovery.html) - client queries a service registry to discover the locations of service instances
- Server-side discovery (server-side-discovery.html) - router queries a service registry to discover the locations of service instances

- Service registry (service-registry.html) - a database of service instance locations
- Self registration (self-registration.html) - service instance registers itself with the service registry
- 3rd party registration (3rd-party-registration.html) - a 3rd party registers a service instance with the service registry

Reliability

How to prevent a network or service failure from cascading to other services?

- Circuit Breaker (reliability/circuit-breaker.html) - invoke a remote service via a proxy that fails immediately when the failure rate of the remote call exceeds a threshold

Security

How to communicate the identity of the requestor to the services that handle the request?

- Access Token (security/access-token.html) - a token that securely stores information about user that is exchanged between services

Observability

How to understand the behavior of an application and troubleshoot problems?

- Log aggregation (observability/application-logging.html) - aggregate application logs
- Application metrics (observability/application-metrics.html) - instrument a service's code to gather statistics about operations
- Audit logging (observability/audit-logging.html) - record user activity in a database
- Distributed tracing (observability/distributed-tracing.html) - instrument services with code that assigns each external request an unique identifier that is passed between services. Record information (e.g. start time, end time) about the work (e.g. service requests) performed when handling the external request in a centralized service
- Exception tracking (observability/exception-tracking.html) - report all exceptions to a centralized exception tracking service that aggregates and tracks exceptions and notifies developers.
- Health check API (observability/health-check-api.html) - service API (e.g. HTTP endpoint) that returns the health of the service and is intended to be pinged, for example, by a monitoring service
- Log deployments and changes (/patterns/observability/log-deployments-and-changes.html)

UI design

How to implement a UI screen or page that displays data from multiple services?

- Server-side page fragment composition (ui/server-side-page-fragment-composition.html) - build a webpage on the server by composing HTML fragments generated by multiple, business capability/subdomain-specific web applications
- Client-side UI composition (ui/client-side-ui-composition.html) - Build a UI on the client by composing UI fragments rendered by multiple, business capability/subdomain-specific UI components

---

🏷 pattern (/tags/pattern)

Modernization Help (https://chrisrichardson.net/)

Copyright © 2026 Chris Richardson • All rights reserved • Supported by Kong (https://konghq.com/).

ALSO ON **MICROSERVICES**

Reducing ...

a year ago · 1 comment

Reducing insider trading in a microservice architecture During a workshop I ...

Microservices ...

2 years ago · 1 comment

Microservices rules: what good looks like - July 2024 edition New public ...

How modular ...

2 years ago · 3 comments

How modular can your monolith go? Part 7 - no such thing as a modular ...

How

2 years

How n
monol
decou

13 Comments

Login

G

Join the discussion...

LOG IN WITH

OR SIGN UP WITH DISQUS

Name

14 Share

Best Newest Oldest

J **Jens Hofmann**
7 years ago

Should there not also be a pattern how to mange code and related files (build, deployment) of microservices? Using a Mono-repository (one for all microservices) or Multi-repository (a repository for each microservice).

22 0 Reply

J **John H**
5 years ago

Nothing on the Bulkhead pattern?

2 1 Reply



Bhuwan Prasad Upadhyay
3 years ago

I think there is small typo mistake "Service component sest". It should be "Service component test"

1 0 Reply

S **Sunil Gora** → Bhuwan Prasad Upadhyay
2 years ago

I want to switch to microservices in nodejs can you please guide me how do i proceed.

0 0 Reply

M **Mitrofan Ius**
a year ago

Why there is no 2PC in patterns?

0 0 Reply

C **carlos henrique alvarenga dos**
a year ago

Very interesting. I am very grateful for your advice.

0 0 Reply

M **Márcio Koch Mk**

4 years ago edited

Hi Chris, what approach do you recommend to use to do a initial load for replicated entity data. This issue happens when a new service is created and it uses a slice of an entity that bellongs to other service that already have data populated and your new service needs this existing data.

0 0 Reply

 **An0nym0usC0ward** → Márcio Koch Mk

3 years ago

Which service is the owner of the data? The owner should be the ultimate authority on that data - and as such, only that service should load it, and every other service should never persistently store that data, just query it from the owning service when needed. If you start replicating data between services (not service instances, mind you), you're opening yourself up for complex failure scenarios where outdated data is being used, or multiple out of sync instances of the same entity are in use at the same time.

That being said, caching is always useful - without caching, microservices-based architectures would fall flat on their nose in any non-trivial scenario, where one operation requires cooperation of multiple services and speedy execution. Caching - specifically cache expiration - is a non-trivial problem not specific to microservices. And it's almost always a compromise between complexity and timeliness requirements. The best compromise is always application-specific.

0 0 Reply

C **Craig Sennett**

4 years ago

will the figure be updated with the new pattern types?

0 0 Reply

 **Simi Hantig**

5 years ago

Hi there! I'm looking to learn how to build multi-tenant microservices. Can you recommend some resources?

0 0 Reply

M **Márcio Koch Mk** → Simi Hantig

4 years ago

Hi Simi, I used RabbitMQ messaging broker with routes including the tenant id as part of the route, and also I added the tenant id inside the message's header. When the message arrives in the service, I put the message context (message data) in a ThreadLocal (Java) to store the current message data, and then switch the data base

schema using the current message tenant.

0 0 Reply



xgqfrms

5 years ago

Awesome